Classification Assignment Evaluation Metrics

RF using GridSearchCV

Classification report:

	precision	recall	f1-score	support
0	0.98	1.00	0.99	51
1	1.00	0.99	0.99	82
accuracy			0.99	133
macro avg	0.99	0.99	0.99	133
weighted avg	0.99	0.99	0.99	133

roc_auc_score:

```
In [18]: from sklearn.metrics import f1_score
         f1_macro=f1_score(y_test,y_pred,average='weighted')
         print('The f1_macro value for best param:{}'.format(gr

         The f1_macro value for best param:{'criterion': 'entro
```

```
In [19]: from sklearn.metrics import roc_auc_score
         roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])

Out[19]: 0.9997608799617408
```

```
id.best_params_), f1_macro)
```

```
py'} 0.9924946382275899
```

Classification Assignment Evaluation Metrics

SVM using GridSearchCV

Classification report :

	precision	recall	f1-score	support
0	0.94	1.00	0.97	51
1	1.00	0.96	0.98	82
accuracy			0.98	133
macro avg	0.97	0.98	0.98	133
weighted avg	0.98	0.98	0.98	133

roc_auc_score:

```
In [34]: from sklearn.metrics import f1_score
         f1_macro=f1_score(y_test,y_pred,average='weighted')
         print('The f1_macro value for best param:{}'.format(gr

         The f1_macro value for best param:{'C': 10} 0.97755569
```

```
In [35]: from sklearn.metrics import roc_auc_score
         roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])

Out[35]: 0.9992826398852224
```

```
id.best_params_), f1_macro)
04684072
```

Classification Assignment Evaluation Metrics

DT using GridSearchCV

Classification report :

	precision	recall	f1-score	support
0	0.91	1.00	0.95	51
1	1.00	0.94	0.97	82
accuracy			0.96	133
macro avg	0.96	0.97	0.96	133
weighted avg	0.97	0.96	0.96	133

roc_auc_score:

```
In [36]: from sklearn.metrics import f1_score
         f1_macro=f1_score(y_test,y_pred,average='weighted')
         print('The f1_macro value for best param:{}'.format(gr

         The f1_macro value for best param:{'splitter': 'random
```

```
In [37]: from sklearn.metrics import roc_auc_score
         roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])

Out[37]: 0.9695121951219512
```

```
id.best_params_), f1_macro)
'} 0.9626932787797391
```

DT

Classification Assignment Evaluation Metrics

Logistic Regression using GridSearchCV

Classification report :

	precision	recall	f1-score	support
0	0.98	1.00	0.99	51
1	1.00	0.99	0.99	82
accuracy			0.99	133
macro avg	0.99	0.99	0.99	133
weighted avg	0.99	0.99	0.99	133

roc_auc_score:

In [15]:
```
from sklearn.metrics import f1_score
f1_macro=f1_score(y_test,y_pred,average='weighted')
print('The f1_macro value for best param:{}'.format(gr
```

```
The f1_macro value for best param:{'solver': 'sag'} 0.
```

In [16]:
```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test,grid.predict_proba(x_test)[:,1])
```

Out[16]: 1.0

```
id.best_params_), f1_macro)
```

```
9924946382275899
```

Classification Assignment Evaluation Metrics

Naive Bayes

GaussianNB Classification report and roc_auc_score :

```
[[51  0]
 [ 3 79]]
              precision    recall  f1-score   support

           0       0.94      1.00      0.97        51
           1       1.00      0.96      0.98        82

    accuracy                           0.98       133
   macro avg       0.97      0.98      0.98       133
weighted avg       0.98      0.98      0.98       133

The f1_macro value 0.9775556904684072
roc_auc_score 1.0
```

MultinomialNB Classification report and roc_auc_score :

```
[[50  1]
 [23 59]]
              precision    recall  f1-score   support

           0       0.68      0.98      0.81        51
           1       0.98      0.72      0.83        82

    accuracy                           0.82       133
   macro avg       0.83      0.85      0.82       133
weighted avg       0.87      0.82      0.82       133

The f1_macro value 0.8215780250262184
roc_auc_score 0.9151123864179818
```

BernoulliNB Classification report and roc_auc_score :

```
[[51  0]
 [ 8 74]]
              precision    recall  f1-score   support

           0       0.86      1.00      0.93        51
           1       1.00      0.90      0.95        82

    accuracy                           0.94       133
   macro avg       0.93      0.95      0.94       133
weighted avg       0.95      0.94      0.94       133

The f1_macro value 0.9404945931261721
roc_auc_score 0.9966523194643712
```

ComplementNB Classification report and roc_auc_score :

```
[[50  1]
 [23 59]]
```

Naive Bayes

```
              precision    recall  f1-score   support

           0       0.68      0.98      0.81        51
           1       0.98      0.72      0.83        82

    accuracy                           0.82       133
   macro avg       0.83      0.85      0.82       133
weighted avg       0.87      0.82      0.82       133

The f1_macro value 0.8215780250262184
roc_auc_score 0.9151123864179818
```

CategoricalNB Classification report and roc_auc_score :

```
[[51  0]
 [ 3 79]]
              precision    recall  f1-score   support

           0       0.94      1.00      0.97        51
           1       1.00      0.96      0.98        82

    accuracy                           0.98       133
   macro avg       0.97      0.98      0.98       133
weighted avg       0.98      0.98      0.98       133

The f1_macro value 0.9775556904684072
roc_auc_score 1.0
```

Classification Assignment Evaluation Metrics

K Nearest Neighbor

Classification report :

	precision	recall	f1-score	support
0	0.86	1.00	0.93	51
1	1.00	0.90	0.95	82
accuracy			0.94	133
macro avg	0.93	0.95	0.94	133
weighted avg	0.95	0.94	0.94	133

roc_auc_score:

```
In [14]: from sklearn.metrics import f1_score
         f1_macro=f1_score(y_test,y_pred,average='weighted')
         print('The f1_macro value', f1_macro)

         The f1_macro value 0.9404945931261721
```

```
In [15]: from sklearn.metrics import roc_auc_score
         roc_auc_score(y_test,classifier.predict_proba(x_test)[:,1])

Out[15]: 0.9996413199426112
```